depends on `EmailServerSettings`, which you'll also need to register with the DI container for your project to run without exceptions.

I avoided registering this object in the preceding example because you have to take a slightly different approach. The preceding `Add*` methods use generics to specify the `Type` of the class to register, but they don't give any indication of *how* to construct an instance of that type. Instead, the container makes several assumptions that you have to adhere to:

- The class must be a concrete type.
- The class must have only a single relevant constructor that the container can use.
- For a constructor to be relevant, all constructor arguments must be registered with the container or must be arguments with a default value.

NOTE These limitations apply to the simple built-in DI container. If you choose to use a third-party container in your app, it may have a different set of limitations.

The `EmailServerSettings` record doesn't meet these requirements, as it requires you to provide a `Host` and `Port` in the constructor, which are a `string` and `int`, respectively, without default values:

```
public record EmailServerSettings(string Host, int Port);
```

You can't register these primitive types in the container. It would be weird to say "For every `string` constructor argument, in any type, use the `"smtp.server.com"` value."